

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)

Assessment Tool Weightage: 10%

QUESTIONS: 10

Total Marks: 20

Annexure A

MOCK INTERVIEW QUESTIONS

Code of Conduct:

I V.Madhulika(192312620) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

V.Madhulika

Signature of the student

Q1. Hospital CSV data with missing values, inconsistent units, duplicates — near real-time processing. How would you clean, filter and standardize it?

Answer:

Sure — first I'd load the data with pandas and quickly profile it using `df.info()` and `isnull().sum()` to see what I'm dealing with. For missing values, I'd fill numeric fields like blood pressure with the median, since it's robust to outliers, and categorical fields with the mode. I'd drop exact duplicates with `drop_duplicates()`, and standardize units — say converting everything to the same weight or temperature unit — with a simple mapping function. For 'near real-time,' I'd process incoming records in small batches rather than reloading the whole file each time, and keep the pipeline vectorized so it stays fast. Finally, I'd save the cleaned output and log any records I had to reject so nothing's silently lost.

Q2. Millions of financial transaction records, need to detect unusual spending patterns under tight memory and time constraints. Your approach?

Answer:

I'd start by using efficient dtypes — `float32` instead of `float64`, `category` dtype for merchant codes — to cut memory right away. Then I'd use `groupby` on customer ID to get spending statistics like mean and standard deviation per customer, and flag transactions that are, say, more than 3 standard deviations away as unusual. I'd avoid Python loops entirely and rely on vectorized NumPy/Pandas operations. If the file's too big for memory, I'd read it in chunks with `chunksize` in `read_csv` and aggregate incrementally. For debugging slow spots, I'd profile with `line_profiler` to find the actual bottleneck rather than guessing.

Q3. Combine sales, marketing, and support datasets from different file formats into one unified DataFrame without losing data or breaking indexing. Walk me through it.

Answer:

First I'd standardize each source's column names so they line up, then read each with the right function — `read_csv`, `read_excel`, `read_json` as needed. I'd merge them on a common key like customer ID using an outer join so I don't lose any records from any department. After merging I'd reset the index to keep it clean and continuous. To validate, I'd check row counts before and after, and use the merge indicator flag to see which rows didn't match on both sides — those get logged for review. Any duplicate or missing keys I'd resolve with a clear priority rule, like trusting the sales system as the source of truth for customer names.

Q4. Build an automated pipeline that reads daily files, filters, computes stats, and produces sorted dashboard outputs, handling corrupted files gracefully. How?

Answer:

I'd loop through the day's files, wrapping each read in a `try/except` so one corrupted file doesn't crash the whole run — I'd just log it and move on. Once loaded, I'd filter to the relevant rows, use `groupby` to compute the summary statistics the dashboard needs, and sort the results so the most important rows show up first. For plotting, I'd generate the chart from the already-aggregated small summary table, not the raw data. And I'd add a schema check at the start of each file read so an unexpected missing column gets caught early with a clear error message instead of failing deep in the pipeline.

Q5. E-commerce clickstream data with missing values and inconsistent formats — how would you preprocess it for downstream ML?

Answer:

Clickstream data is usually event-based, so I'd first make sure each event has a consistent timestamp format using `pd.to_datetime`. For missing values — like a missing page category — I'd fill with a placeholder like 'unknown' rather than dropping the row, since click data is valuable even if incomplete. I'd extract features like session length, number of clicks, and time between actions using `groupby` on session ID. For indexing, I'd set a session-level index so it's easy to join back to user features later. The end goal is a clean, feature-rich table ready to hand off to a model.

Q6. Healthcare vitals as time-series data with irregular timestamps and noisy readings — how would you clean, resample, and analyze it?

Answer:

I'd set the timestamp as the index, sort it chronologically, then resample to a fixed interval, like every 5 minutes, using the mean. For irregular gaps, I'd interpolate short ones using time-based interpolation, but leave longer gaps as missing rather than making up data. For noise, I'd apply a rolling average to smooth the signal, and use rolling statistics — like a rolling z-score — to flag abnormal spikes, which is exactly what you want for detecting things like a sudden heart rate spike.

Q7. Millions of rows with mixed data types and categoricals, prepping for ML under strict time constraints — how would you encode, normalize, and transform?

Answer:

I'd convert dtypes efficiently first — numeric columns downcast, categorical columns as category dtype — since that alone saves a lot of memory and speeds things up. For encoding, I'd use one-hot encoding for low-cardinality categories and something like frequency or target encoding for high-cardinality ones, since one-hot would blow up the column count otherwise. Normalization I'd do with vectorized Z-score or min-max scaling. To hit the time constraint, I'd process everything in chunks and avoid row-wise `apply()` functions, which are a common hidden performance killer in pandas.

Q8. Logistics company analyzing delivery performance with location, time, and traffic data — how would you design the pipeline?

Answer:

I'd group by route or region and aggregate metrics like average delivery time and on-time percentage. Filtering would come first — dropping test or cancelled orders before aggregating. I'd set a proper index, probably a composite of route and date, so lookups are fast, and sort by delivery time or delay to surface the worst-performing routes at the top. To validate accuracy, I'd spot-check a few aggregated numbers against the raw records manually, since that catches subtle grouping mistakes that automated tests might miss.

Q9. Build a data validation system for incoming ML pipeline data — checking missing values, invalid ranges, inconsistent formats. How?

Answer:

I'd define a schema upfront — expected data type and valid range for each column — and run every incoming batch through a validation function that checks each rule and logs any violations to a report rather than just crashing. I'd use boolean filtering to separate valid rows from invalid ones automatically, and use aggregation to summarize how many rows failed each rule, which makes it easy to spot a systemic upstream issue versus a one-off bad record. The valid rows go through to the pipeline; the rest go into a quarantine file for review.

Q10. Analyze user activity logs for peak usage times and segmentation on a large, continuously updated dataset — how would you approach it?

Answer:

I'd bucket timestamps into hourly or daily intervals and use groupby plus count to find peak usage times — that's a straightforward aggregation once the timestamp is parsed correctly. For segmentation, I'd group users by behavior metrics like session frequency or average session length, maybe using simple clustering after normalizing those features. Since it's continuously updated, I'd process new data incrementally — append and re-aggregate only the new batch — rather than recomputing everything from scratch each time, which keeps it scalable.

Q11. Small dataset of student records with missing marks and inconsistent grade formats. How would you clean it, in simple steps?

Answer:

Pretty straightforward one — I'd load it with pandas, check which rows have missing marks using `isnull()`, and fill those with the class average or median. For inconsistent grades — like 'A', 'a', 'Excellent' all meaning the same thing — I'd write a small mapping dictionary and apply it to standardize every value to one consistent format. Then I'd double check with `unique()` that only the expected grade values remain before saving the cleaned file.

Q12. Find total and average sales per product category using Pandas grouping and aggregation. Explain the steps.

Answer:

That's a classic groupby task. I'd do `df.groupby('category')['sales'].agg(['sum','mean'])`, which groups all rows by category and computes both metrics in one pass. Then I'd sort the result by total sales descending so the top categories are easy to see, and that's basically the whole answer — pandas' groupby-aggregate combo handles it in one or two lines.

Q13. Dataset has duplicate customer entries and numbers stored as strings. How would you identify and fix these?

Answer:

For duplicates, I'd use `duplicated()` to see how many there are before deciding, then `drop_duplicates()` on the customer ID, keeping the most recent entry. For the string-numbers issue, I'd use `pd.to_numeric()` with `errors='coerce'`, which converts valid numbers and turns anything unconvertible into NaN so I can catch and fix those separately rather than have them silently break downstream calculations.

Q14. Timestamps aren't sorted; you need to sort them and compute a simple moving average. How would you do it?

Answer:

First I'd convert the timestamp column with `pd.to_datetime`, then `sort_values('timestamp')` to get everything chronological. Once sorted, the moving average is just one line — `df['value'].rolling(window=n).mean()` — pandas handles the sliding window calculation for you, so there's no need to write that logic manually.

Q15. Dataset with numerical and categorical features for an ML model — how would you handle missing values, encode categories, and normalize numbers, briefly?

Answer:

Three quick steps: fill missing numeric values with the median and missing categorical values with the mode or an 'unknown' label; encode categorical columns with one-hot encoding using `pd.get_dummies()`; and normalize the numeric columns with `StandardScaler` or a simple min-max formula so all features are on a comparable scale before feeding them into the model.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	25	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand